CSCI 496: Introduction to Cloud Computing

Project: AI-Based Smart Attendance System Using Face Recognition

Name: Muhammadjon Merzakulov

Student ID: 600002775

Instructor: Umair Arif

Submission date: 22.04.2026

Table of Contents

Introduction

Attendance tracking is a critical administrative procedure within educational institutions and corporate enterprises. Traditional methodologies, such as manual roll calls, paper-based registers, or physical ID card swiping, are highly inefficient, time-consuming, and susceptible to human error and fraudulent practices (e.g., proxy attendance).

This project introduces AttendAI, an intelligent, touchless Face Recognition Attendance System designed to modernize and automate identity verification. Engineered utilizing a robust, modern technology stack comprising Django, Celery, DeepFace, Docker, and Microsoft Azure, the system captures real-time biometric data and automatically verifies user identities. By integrating secure QR Code workflows, utilizing Tailwind CSS for a responsive frontend, and employing role-based asynchronous background processing, AttendAI ensures high accuracy, cryptographic security, and a seamless user experience without causing computational bottlenecks on the main web server.

Problem Statement and Motivation

Problem Statement: Traditional attendance systems suffer from several critical limitations:

- **Time Inefficiency:** Manual attendance processes consume valuable time during lectures or working hours.
- **Lack of Security:** Proxy attendance (marking attendance for absent individuals) is common and difficult to prevent.
- **Poor Data Management:** Paper-based records are difficult to maintain, update, and analyze in real time.

Motivation: The motivation behind this project is to design a system that is automated, secure, and scalable. By leveraging Artificial Intelligence and Cloud Computing technologies, the system aims to eliminate manual intervention and improve reliability. Additionally, this project provides practical experience in integrating AI models with web applications and deploying them using modern containerization tools.

Objectives

The primary objectives of this project are:

- To develop a fully automated and contactless attendance system.
- To implement accurate facial recognition using the DeepFace framework.
- To enhance security through QR code-based identity verification.
- To ensure system responsiveness by offloading computational tasks using Celery and Redis.
- To deploy the system using Docker for portability and scalability.

Methodology / System Development

Development Process: The system was developed incrementally using a modular approach:

1. Designing the database schema and backend architecture.
2. Developing the frontend interface for image capture.
3. Integrating the DeepFace AI model for facial recognition.
4. Implementing asynchronous task processing using Celery and Redis.
5. Packaging and deploying the system using Docker on Azure

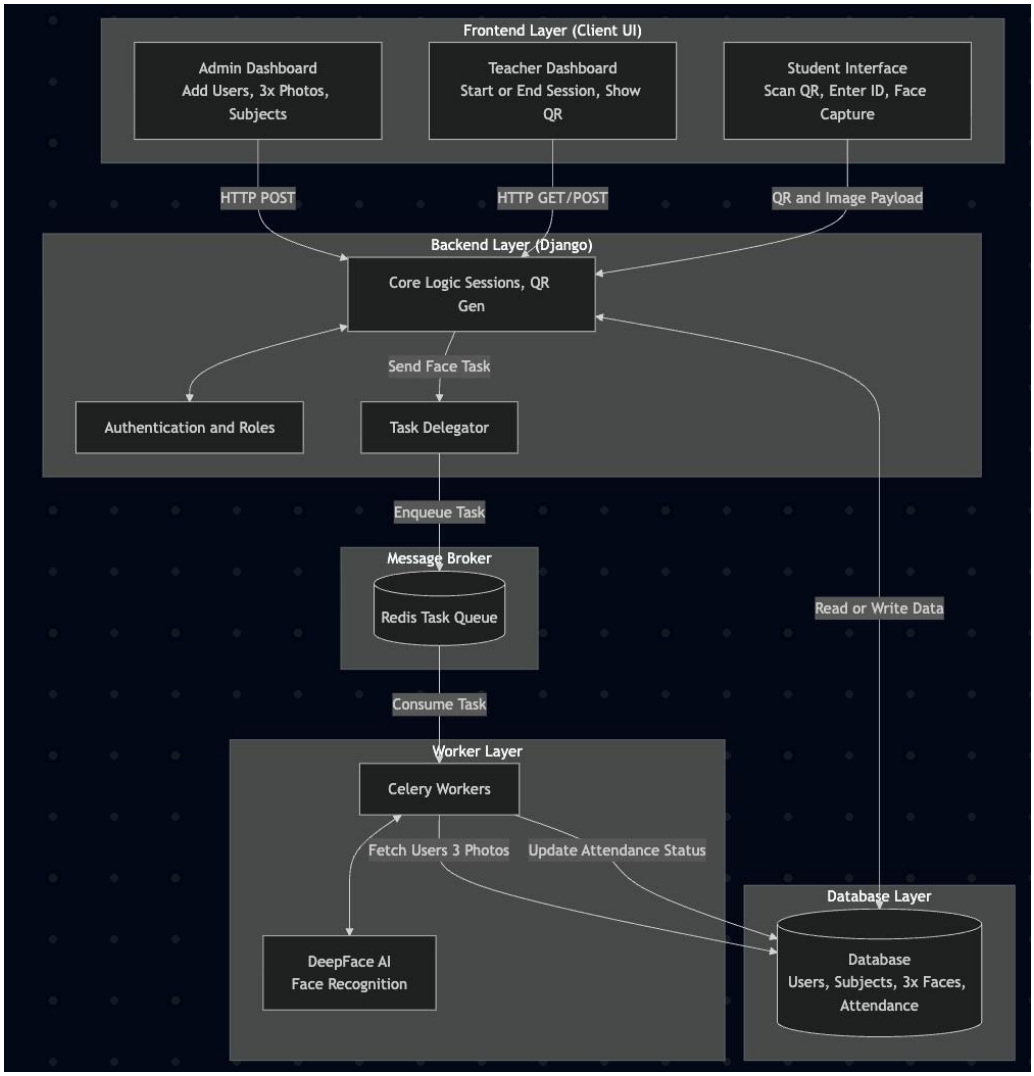
Technologies and Tools

- **Frontend:** HTML, Tailwind CSS, JavaScript,
- **Backend:** Python with Django framework
- **AI Model:** DeepFace
- **Task Queue:** Celery with Redis message broker
- **Deployment:** Docker and Microsoft Azure
- **Database:** PostgreSQL

System Architecture

The system follows a distributed architecture consisting of the following components:

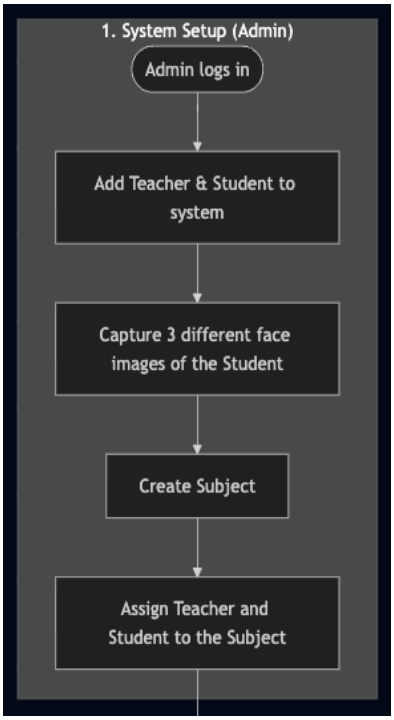
- **Frontend Layer:** Provides a user interface for QR code scanning and image capture.
- **Backend Layer (Django):** Handles HTTP requests, user authentication, and task delegation.
- **Message Broker (Redis):** Queues tasks and ensures reliable communication between services.
- **Worker Layer (Celery):** Processes computationally intensive facial recognition tasks.
- **Database Layer:** Stores user data and attendance records.

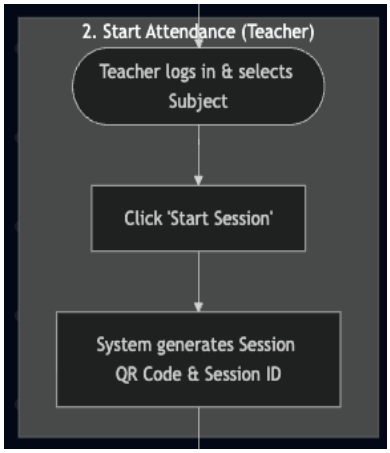


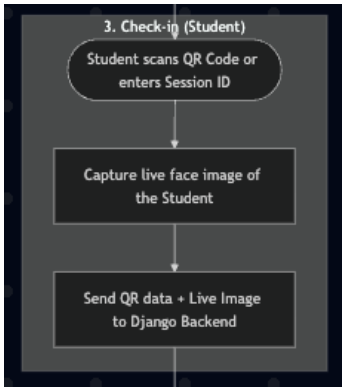
Flowchart / Working Pipeline

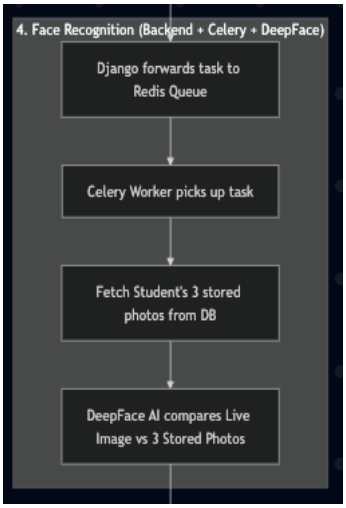
The system operates through the following sequence:

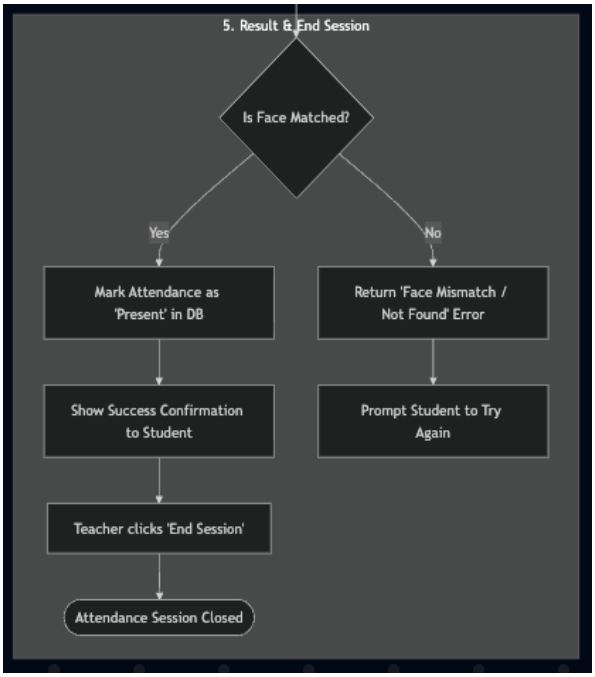
1. The user presents a QR code to the system.
2. The system scans the QR code and captures a facial image.
3. The image is transmitted to the Django backend.
4. The backend forwards the task to the Celery queue.
5. The DeepFace model performs facial recognition.
6. If a match is found, attendance is recorded in the database.
7. The user receives a confirmation message.











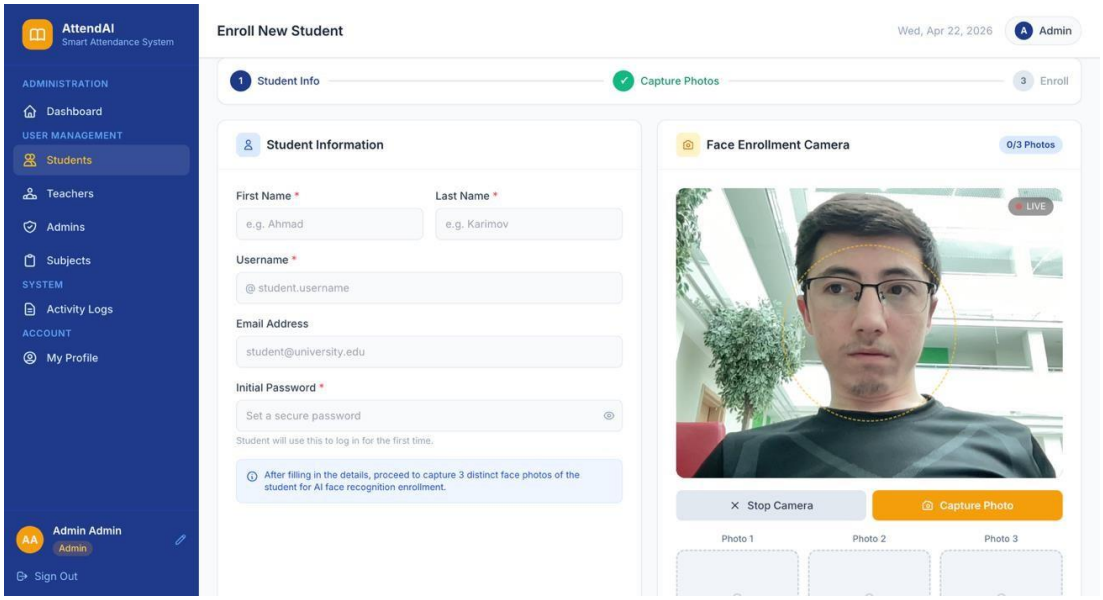
Implementation Details

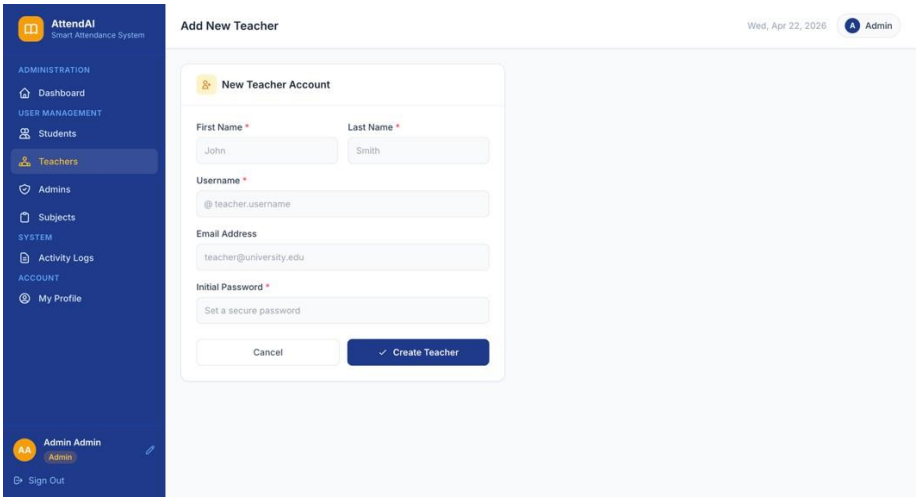
- **Frontend Implementation:** Built using HTML, JavaScript, and Tailwind CSS to create a responsive and user-friendly interface. The browser camera is used for image capture.
- **Backend Implementation:** Django manages request handling, authentication, and database operations.
- **AI Integration:** DeepFace is used for facial recognition. Due to its computational complexity, tasks are processed asynchronously.
- **Asynchronous Processing:** Celery and Redis are used to execute heavy AI tasks in the background, preventing system delays.
- **Deployment:** Docker is used to package the application and ensure consistent performance across environments. The system is deployed on Microsoft Azure for cloud accessibility.

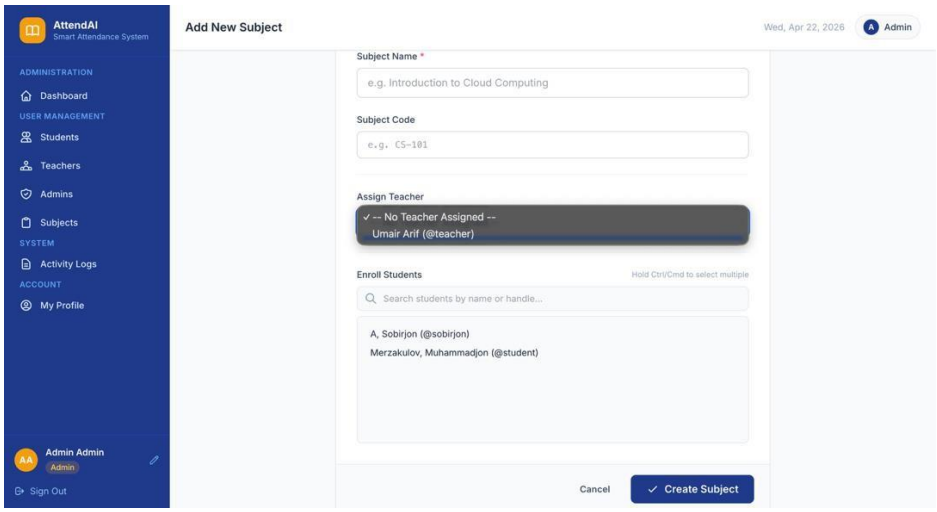
Results

The system was successfully implemented, tested, and deployed on Microsoft Azure. It demonstrates:

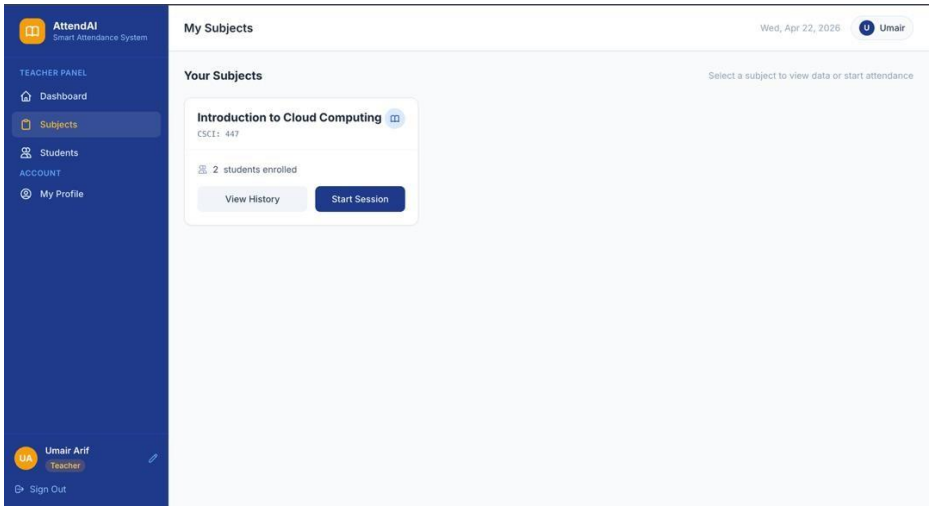
Admin panel

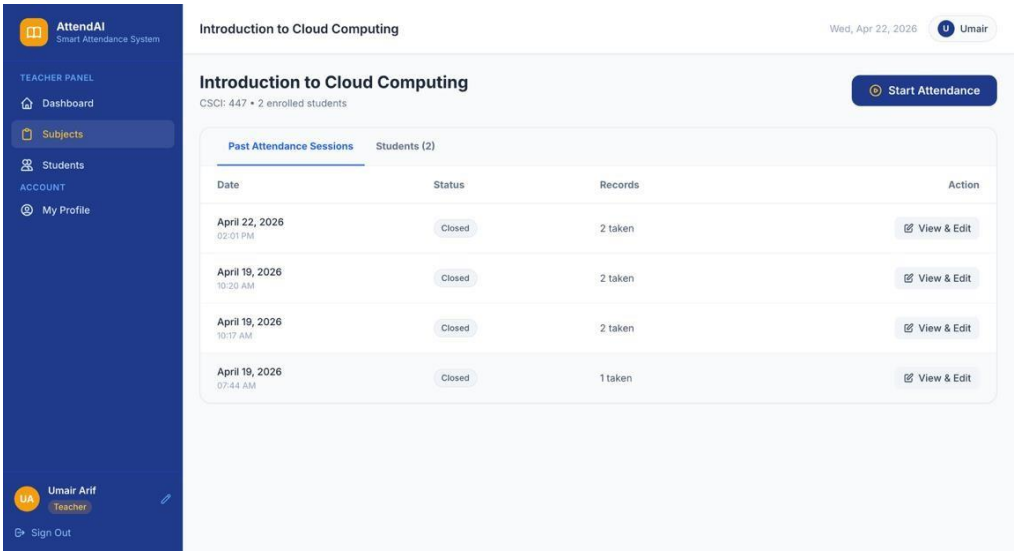


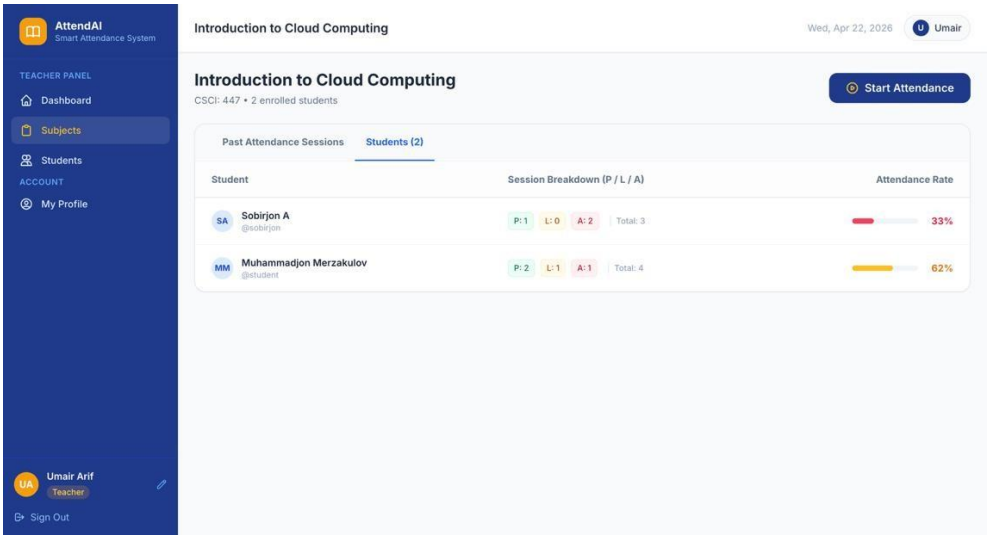


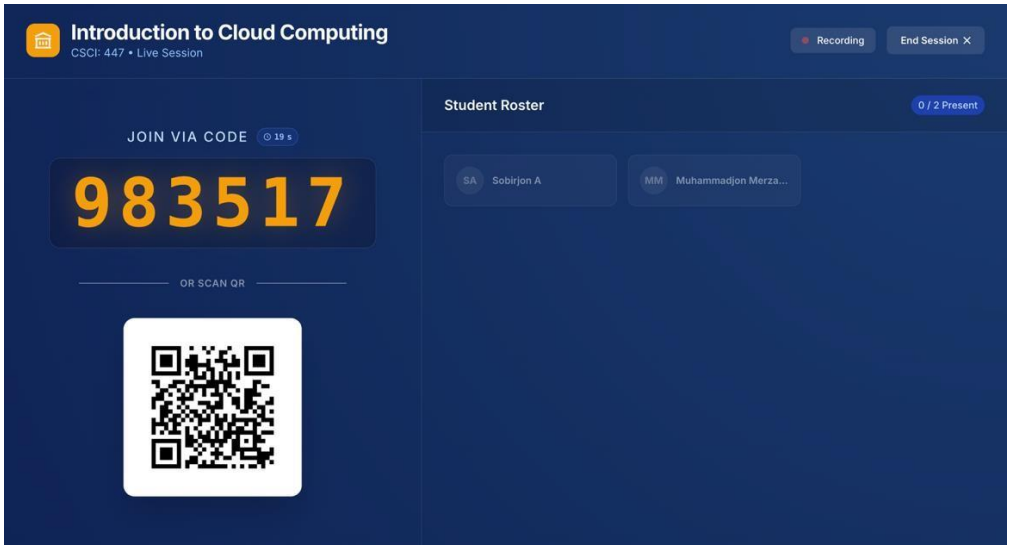


Teacher panel

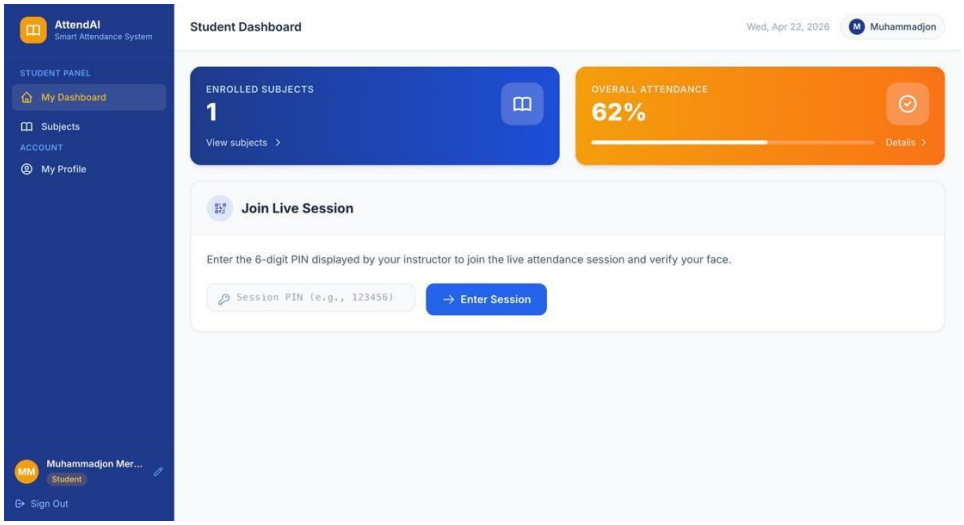


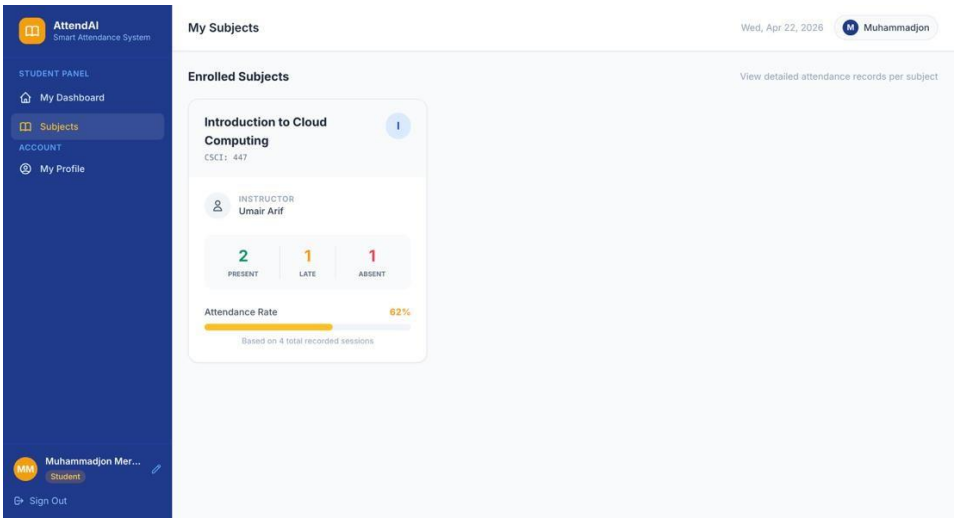

AttendAI
Smart Attendance System
TEACHER PANEL
Dashboard
Subjects
Students
ACCOUNT
My Profile
Umair Arif
Teacher
Sign Out
Introduction to Cloud Computing
Wed, Apr 22, 2026
Umair
Introduction to Cloud Computing
CSCI: 447 • 2 enrolled students
Start Attendance
Past Attendance Sessions
Students (2)
Date
Status
Records
Action
April 22, 2026
02:01 PM
Closed
2 taken
View & Edit
April 19, 2026
10:20 AM
Closed
2 taken
View & Edit
April 19, 2026
10:17 AM
Closed
2 taken
View & Edit
April 19, 2026
07:44 AM
Closed
1 taken
View & Edit

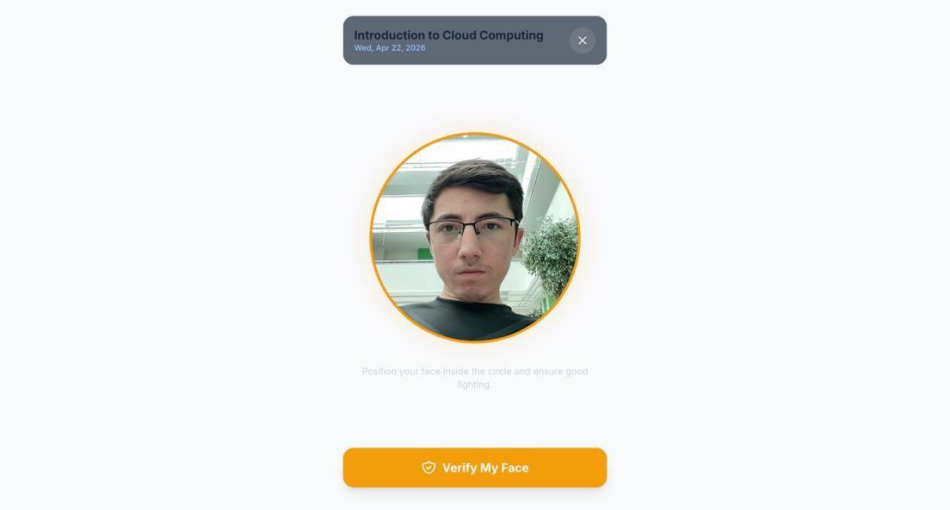

AttendAI
Smart Attendance System
TEACHER PANEL
Dashboard
Subjects
Students
ACCOUNT
My Profile
Umair Arif
Teacher
Sign Out
Introduction to Cloud Computing
Wed, Apr 22, 2026
Umair
Introduction to Cloud Computing
CSCI: 447 • 2 enrolled students
Start Attendance
Past Attendance Sessions
Students (2)
Student
Session Breakdown (P / L / A)
Attendance Rate
SA
Sobirjon A
@sobirjon
P: 1
L: 0
A: 2
Total: 3
33%
MM
Muhammadjon Merzakulov
@student
P: 2
L: 1
A: 1
Total: 4
62%

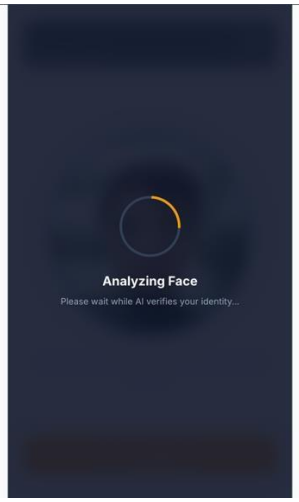

Introduction to Cloud Computing
CSCI: 447 • Live Session
Recording
End Session
JOIN VIA CODE
19 s
983517
OR SCAN QR
Student Roster
0 / 2 Present
SA
Sobirjon A
MM
Muhammadjon Merza...

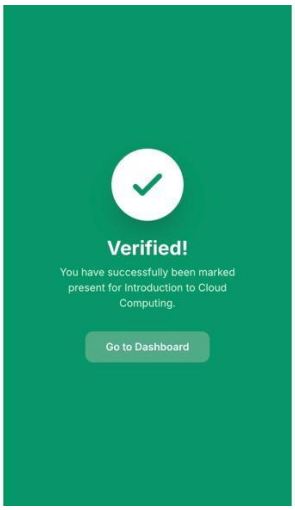
Student Panel



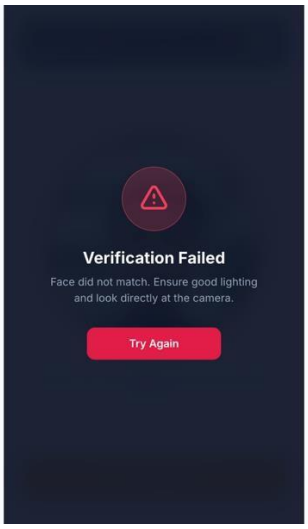





Analyzing Face
Please wait while AI verifies your identity...


Verified!
You have successfully been marked present for Introduction to Cloud Computing.
Go to Dashboard

or

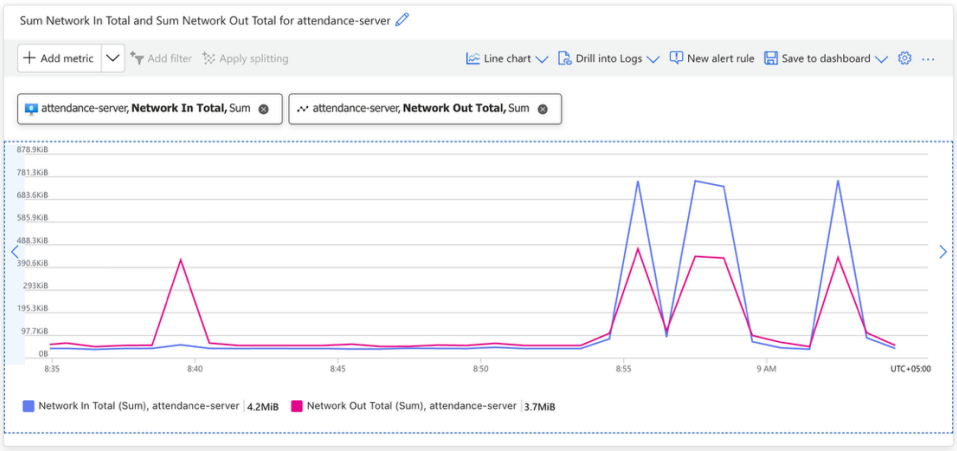

Verification Failed
Face did not match. Ensure good lighting and look directly at the camera.
Try Again

Cloud Computation and Network Metrics Evaluation

To evaluate the system's performance and stability on the cloud, hardware and network metrics of the Azure Virtual Machine (Ubuntu 24.04) were monitored during student registration and facial verification sessions

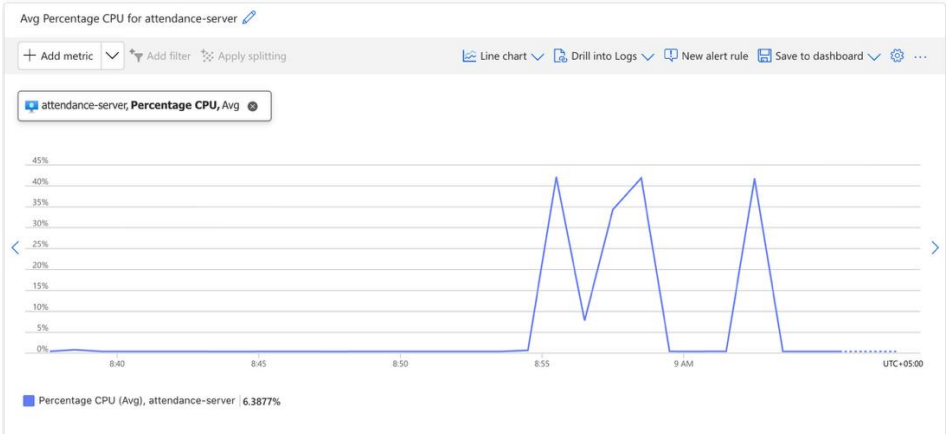
A. Infrastructure Metrics (Azure VM Level)

1. Network Throughput (Bandwidth Efficiency)



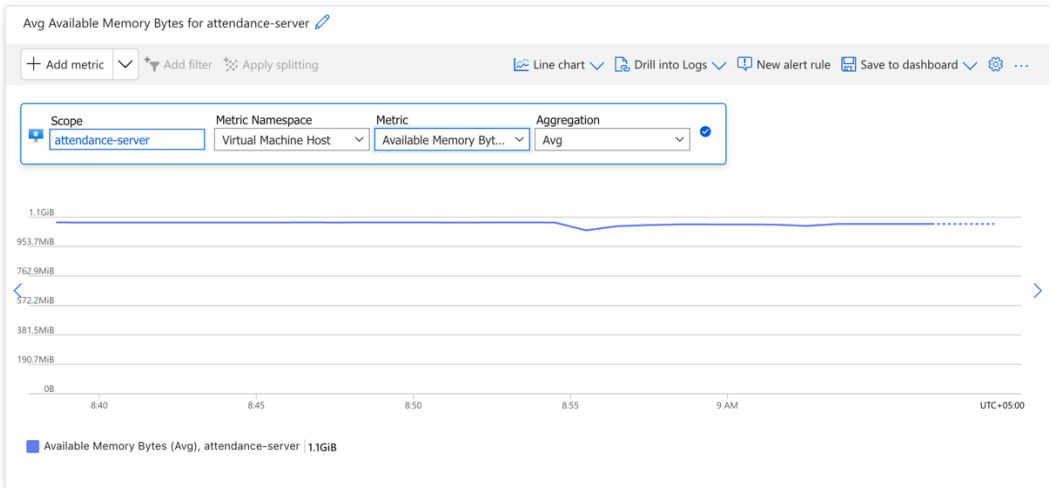
- **Analysis**: The graph illustrates total network traffic during peak activities at 8:55, 8:58, and 9:02.
- **Observation**: Even during multiple concurrent student enrollments and face scans, the peak incoming traffic (Network In) reached only **~780 KiB**.
- **Conclusion**: This indicates highly efficient bandwidth utilization. By transmitting lightweight **Base64** strings instead of raw image files, the architecture prevents network bottlenecks.

2. Computation: CPU Utilization



- **Analysis**: Since the Azure VM (Standard B1s) does not utilize a dedicated GPU, facial recognition is a CPU-intensive operation.
- **Observation**: At baseline, CPU usage is near 0%. During face extraction and matching, CPU utilization spiked predictably to **~40-45%** and immediately returned to baseline.
- **Conclusion**: The asynchronous architecture using **Celery and Redis** successfully offloads heavy AI workloads. The main Django web server remained entirely unblocked and responsive throughout the spikes.

3. Computation: Memory (RAM) Stability

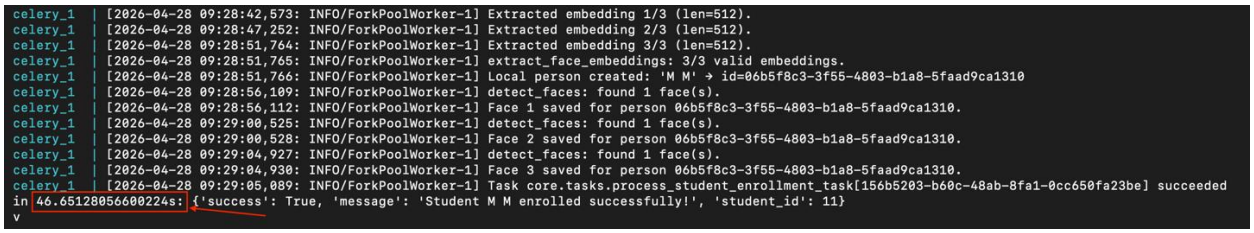


- **Analysis**: Significant memory is required to load pre-trained AI weights for the DeepFace framework.
- **Observation**: The available memory remained highly stable at **~1.1 GiB** during the testing period.
- **Conclusion**: There are no memory leaks detected in the system. The Docker containers manage memory efficiently, ensuring long-term operational stability.

B. Application-Level Metrics (Logs & Timings)

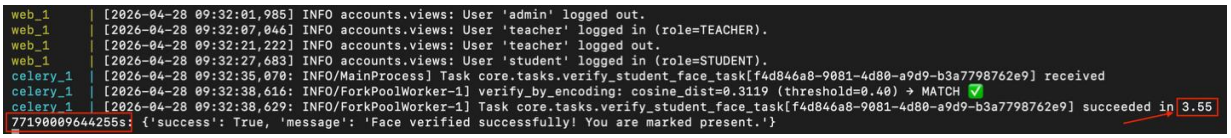
Application-specific latency and execution times were measured directly via Python loggers within the **Celery worker**.

1. The "Cold Start" Penalty (Initial Enrollment)



- **Analysis**: During the initial execution of the *process_student_enrollment_task*, the computation took **46.65** seconds.
- **Reason**: This represents the "Cold Start" period where the DeepFace engine initializes and downloads pre-trained weights into the Docker volume for the first time.
- **Impact**: Because this is a background task handled by Celery, this **46-second** delay did not freeze or impact the user interface.

2. "Warm Start" Performance (Real-time Verification)



- **Analysis**: For subsequent facial verifications (*verify_student_face_task*), the execution time dropped significantly to just **3.55 seconds**.
- **Reason**: With the AI models now cached in the VM's RAM, the system bypassed I/O overhead.
- **Mechanism**: The system extracts a 512-dimensional embedding and calculates the **Cosine Distance**. The log confirms a *cosine_dist=0.3119*, which successfully passes the *< 0.40* threshold for a match.

Challenges Faced

During the development and deployment of the *Attend AI* system, several technical challenges were encountered and successfully resolved:

- **Webcam Access Blocked After Cloud Deployment:** *Challenge*: After deploying the system to the Microsoft Azure cloud, the frontend failed to open the user's webcam. Modern web browsers strictly block access to the camera API (*navigator.mediaDevices.getUserMedia*) when a website is loaded over an insecure HTTP connection. *Solution*: Configured SSL/TLS certificates to secure the Azure deployment with HTTPS. Once the connection was encrypted, the browsers safely granted camera permissions, allowing the face capture feature to function flawlessly.
- **Performance Issues and System Delays:** *Challenge*: The initial synchronous implementation caused severe system delays and server timeouts due to the heavy computational load of the DeepFace AI models running directly within the HTTP request cycle. *Solution*: Integrated Celery and Redis to create an asynchronous task queue. This allowed the heavy facial recognition processing to be executed in the background, ensuring the frontend interface remained fast and highly responsive.
- **Environmental Constraints (Lighting Conditions):** *Challenge*: Facial recognition accuracy noticeably decreased when users captured their photos under poor, dim, or uneven lighting conditions. *Solution*: Made fine-tuned adjustments to the DeepFace model's confidence threshold parameters and added UI guidance to instruct users on capturing better-lit, clear images for accurate recognition.

Conclusion

In conclusion, the *AttendAI* project successfully transforms the traditional attendance process into a secure, contactless, and fully automated system. By integrating Django for a reliable backend and Tailwind CSS for a responsive frontend, the system delivers a clean, intuitive, and user-friendly experience. The use of DeepFace as the core biometric engine enables accurate facial recognition, effectively reducing manual errors and preventing fraudulent practices such as proxy attendance.

A key technical contribution of this project is the efficient handling of computationally intensive AI tasks. Through the use of Celery and Redis, facial recognition processes are executed asynchronously in the background, ensuring that the main application remains fast, stable, and responsive. Furthermore, containerization with Docker and deployment on Microsoft Azure demonstrate the system's scalability, portability, and readiness for real-world environments.

Overall, AttendAI successfully achieves its objectives by combining artificial intelligence, modern web technologies, and cloud-based deployment. The project highlights how advanced technologies can be practically applied to improve efficiency, security, and reliability in administrative systems.

Future Work

Potential improvements include:

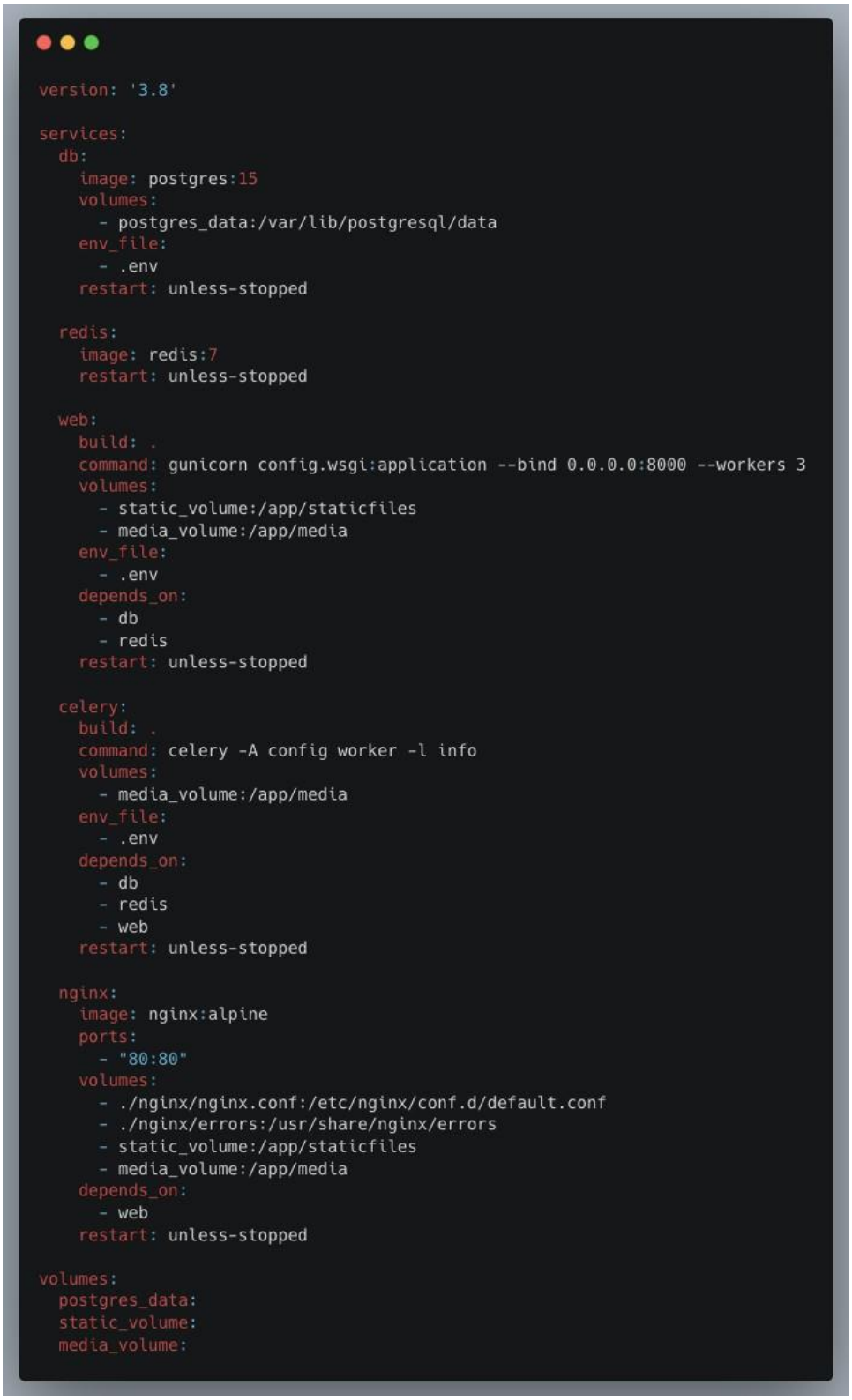
- **Liveness Detection:** Preventing spoofing using printed images or videos.

References

1. Source Code (GitHub): https://github.com/muhammadjon-merzaqulov/face-based-attendance-system
2. Live Application (Deployed System): https://face.muhammadjon.uz
3. Project Demonstration Video: https://youtu.be/bs_P-LlqLME

Appendix

My ***docker-compose.yml*** configuration:



```yaml
version: '3.8'

services:
  db:
    image: postgres:15
    volumes:
      - postgres_data:/var/lib/postgresql/data
    env_file:
      - .env
    restart: unless-stopped

  redis:
    image: redis:7
    restart: unless-stopped

  web:
    build: .
    command: gunicorn config.wsgi:application --bind 0.0.0.0:8000 --workers 3
    volumes:
      - static_volume:/app/staticfiles
      - media_volume:/app/media
    env_file:
      - .env
    depends_on:
      - db
      - redis
    restart: unless-stopped

  celery:
    build: .
    command: celery -A config worker -l info
    volumes:
      - media_volume:/app/media
    env_file:
      - .env
    depends_on:
      - db
      - redis
      - web
    restart: unless-stopped

  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf
      - ./nginx/errors:/usr/share/nginx/errors
      - static_volume:/app/staticfiles
      - media_volume:/app/media
    depends_on:
      - web
    restart: unless-stopped

volumes:
  postgres_data:
  static_volume:
  media_volume:
```

Azure portal